Final Project:
Sportlytics Athletics Professional Basketball Analytics

Team BigQuery Quest
Marta Falceto Font
Kadiatou Sogodogo
Karthikeyan Atmakuru

Under the Guidance of
Dr. Tim Smith

University of South Florida
Muma College of Business

ISM6562 – Big Data For Business

Table of Contents

1. **Executive Summary** 3
   - **1.1 Business Problem:** 3
   - **1.2 Key Findings:** 3
2. **Architecture Design** 4
   - **2.1 Pipeline architecture diagram** 4
   - **2.2 Technology Choices and Justifications** 5
   - **2.3 Full Data Flow: Landing to Curated to Analytics** 5
   - **2.4 Lambda versus Kappa Architecture Decision** 6
   - **2.5 Docker Infrastructure** 6
3. **Implementation** 6
   - **3.1 Key Technical Decisions:** 6
     - **Stage 1: Data Lake Foundation** 6
     - **Stage 2: Spark batch transformation** 7
     - **Stage 3: Real-Time Streaming Layer** 7
     - **Stage 4: Pipeline Orchestration** 8
   - **3.2 Challenges and Solutions:** 8
   - **3.3 Kafka topic design** 10
   - **3.4 Airflow DAG Structure** 11
     - **Batch pipeline DAG (batch_pipeline.py)** 11
   - **3.5 Data Quality Strategy** 12
4. **Business Insights** 12
   - **Recommendation** 16
5. **Scalability Discussion** 17
   - **5.1 Bottleneck Analysis** 17
   - **5.2 Specific Bottlenecks** 17
     - **What breaks first at 10x:** 17
     - **What breaks first at 100x:** 18
   - **5.3 Cloud Migration and Scalability Path** 18

1. Executive Summary

Sportlytics Analytics is the analytics partner for a professional basketball league consisting of 30 teams and over 450 active players. The current pipeline, initially developed as a three team proof of concept, has reached its operational limits. Nightly batch processes fail during peak playoff data volumes, injury prediction models require approximately 14 hours to retrain and the in-game dashboard experiences a 45-second latency, which makes it ineffective for real-time substitution decisions. As a result, three teams have indicated that they may terminate their contracts.

BigQuery Quest has designed a comprehensive solution to address these challenges. The approach adopts a Lambda Architecture across these key layers: an HDFS data lake, Spark-based batch processing, Kafka-enabled real-time streaming and Airflow-driven orchestration. The redesigned system supports five seasons of historical tracking data, identifies injury-risk players up to seven days in advance, quantifies performance decline and delivers personalized fatigue alerts to coaching staff in under five seconds during live games.

1.1 Business Problem:

Sportlytics' current system limitations are most visible in a few key areas. These form the focus of the new pipeline redesign:

- Player injury costs: Teams collectively lost about $780M in player salary due to time spent on the injured list last season. The current injury prediction model only identifies 38% of cases, meaning it misses nearly two-thirds of injuries that could potentially be prevented.
- In-game latency: The current tracking data pipeline has a 45-second delay during live games. Coaches need alerts in under 5 seconds when player workload reaches fatigue thresholds to make timely substitution decisions.
- Fragmented data: Practice load, recovery sessions, sleep data, travel schedules, and game performance have not been integrated into a single analytical system.

1.2 Key Findings:

While league-wide workload management prevents cumulative fatigue from being noticeable in overall metrics, significant performance differences still appear at the individual player level. BigQuery Quest identified the following key findings:

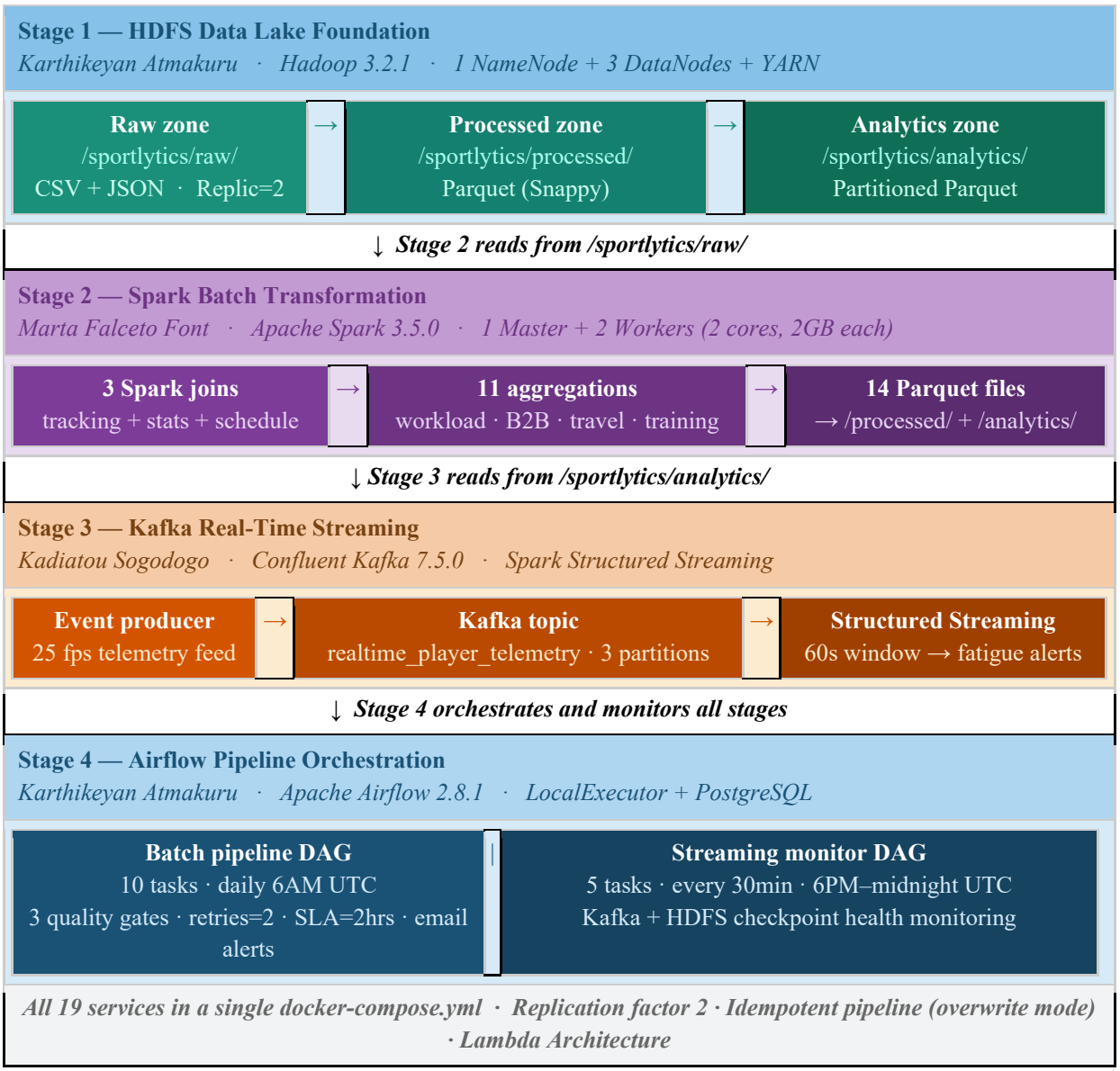
- High-usage players show the most significant back-to-back fatigue impact, with plus-minus declining from +0.021 on normal rest to -0.074 on consecutive nights — a drop that coaching staff can now anticipate and manage proactively using workload profiles computed nightly by the batch pipeline.
- The optimal training protocol for maintaining peak game performance is approximately 1.5 practice sessions per recovery session per week. Players in the moderate practice-to-recovery ratio (1–2:1) outperformed all other groups with a plus-minus of +0.474 and shooting percentage of 50.83%.

- Rest days are a stronger performance predictor than travel distance. Games following four or more rest days produce the strongest league-wide performance (+0.089), regardless of how far teams traveled.
- The real-time streaming pipeline processed 187,706 sixty-second windows and triggered 12,508 personalized alerts, all within sub-five-second latency. This replaces the previous 45-second delay that made in-game monitoring ineffective for substitution decisions.

2. Architecture Design

2.1 Pipeline architecture diagram

The Sportlytics big data pipeline is built across four stages following a Lambda Architecture pattern, combining a batch layer for historical analysis with a speed layer for real-time fatigue alerting:



2.2 Technology Choices and Justifications

The pipeline uses nine core technologies, each chosen for a specific reason. Apache Hadoop HDFS 3.2.1 provides distributed fault-tolerant storage and separates storage from compute so each can scale independently. Apache Spark 3.5.0 handles in-memory distributed processing with native HDFS and Parquet integration and a unified API for both batch and streaming. Apache Kafka 7.5.0 (Confluent) serves as the high-throughput message queue for 25 fps telemetry, with 3 partitions to maintain per-player event ordering. Apache Airflow 2.8.1 provides DAG-based scheduling with quality gates, retry logic, SLAs, and email failure alerting.

Supporting infrastructure includes Apache Zookeeper 7.5.0 for Kafka broker coordination, and PostgreSQL 13 as the Airflow metadata database. For file formats, CSV and JSON are preserved in the raw zone as an immutable source of truth, while Parquet with Snappy compression is used in the processed and analytics zones for columnar storage and 60 to 70 percent compression versus raw CSV. All 19 services are defined in a single Docker Compose file for single-command reproducible setup across all four stages.

2.3 Full Data Flow: Landing to Curated to Analytics

Landing zone: /sportlytics/raw/

- 5 raw files ingested via scripts/01_load_hdfs.sh: player-tracking.csv (55.8 MB), game-stats.json (72 MB), injury-reports.csv (0.9 MB), training-sessions.json (28.5 MB), team-schedules.csv (0.7 MB)
- Original formats preserved: raw zone is the immutable source of truth with no transformation risk at ingestion
- Replication factor 2 across 3 DataNodes: tolerates 1 node failure without data loss
- Both compressed (.gz) and uncompressed versions loaded for Stage 2 compatibility

Curated zone: /sportlytics/processed/

- tracking_with_stats.parquet: player spatial data joined with game box score outcomes per player per game
- training_with_schedule.parquet: training sessions enriched with game schedule context
- tracking_stats_schedule.parquet: full enriched dataset combining all three sources
- All written by Stage 2 Spark in overwrite mode: idempotent and safe to re-run without duplicates

Analytics zone: /sportlytics/analytics/

- rolling_workload.parquet: 5-game rolling distance, heart rate, and minutes per player (injury risk input)
- back_to_back_analysis.parquet: performance metrics on back-to-back versus normal rest games
- travel_rest_analysis.parquet: performance by rest days and travel distance combinations
- training_load_analysis.parquet: intensity, sleep, and duration by session type
- streaming_alerts.parquet: real-time fatigue alerts written by Stage 3 Structured Streaming

2.4 Lambda versus Kappa Architecture Decision

We chose Lambda Architecture over Kappa because Sportlytics requires two fundamentally different types of processing. The batch layer handles complex multi-season rolling window aggregations for injury risk modeling, where re-computing full historical seasons on every new streaming event would be prohibitively expensive. The speed layer handles only lightweight per-window aggregations for live fatigue alerts, where sub-60-second latency is the priority.

Kappa architecture was rejected because a single streaming pipeline cannot efficiently join large historical datasets or compute the multi-season rolling features needed for injury prediction. Lambda gives us the best of both: rich historical analysis in batch and fast real-time alerting in the speed layer, with the HDFS analytics zone serving as the shared serving layer for both.

2.5 Docker Infrastructure

All 19 services are organized across four stages in a single docker-compose.yml file. Stage 1 runs namenode, three DataNodes, ResourceManager, three NodeManagers, and the History Server on ports 9870, 9000, 8088, 8042 to 8044, and 8188. Stage 2 adds spark-master, two Spark workers, and Jupyter on ports 8080, 7077, 4040, 8081, 8082, and 8888. Stage 3 adds Zookeeper, Kafka, and Kafka-UI on ports 2181, 9092, 29092, and 8089. Stage 4 adds PostgreSQL, airflow-init, airflow-webserver, and airflow-scheduler on port 8085.

LocalExecutor was chosen for Airflow over CeleryExecutor because the pipeline runs on a single machine, avoiding the Redis and RabbitMQ dependency and reducing memory overhead. WSL2 .wslconfig was configured to allocate 12 GB RAM to Docker to support the full stack within the course environment.

3. Implementation

BigQuery Quest built Sportlytics new pipeline through iterative development, encountering and resolving real engineering challenges at every stage.

3.1 Key Technical Decisions:

Stage 1: Data Lake Foundation

- **Descriptive Zone Naming Over Generic Terminology:** We named the three HDFS zones raw, processed, and analytics rather than landing, curated, and analytics. The names more clearly communicate data state to the analytics team and reduce onboarding friction. The zones serve the same architectural purpose as the standard medallion pattern.
- **Preserving Original Formats in the Raw Zone:** All five datasets were stored in their original formats: CSV for tabular files and JSON for nested structures. Converting to Parquet at ingestion would introduce transformation risk. The raw zone serves as the immutable source of truth, allowing the pipeline to be re-run from scratch if any downstream stage produces corrupt output.
- **Replication Factor 2 over the Hadoop Default of 3** With exactly 3 DataNodes, a replication factor of 3 writes every block to every node, eliminating load-balancing benefits

and tripling storage usage. Factor 2 tolerates a single DataNode failure while keeping meaningful headroom. In production, legally sensitive injury records would be selectively replicated at factor 3.

- **Partitioning by team_id and game_id for Query Performance** Processed and analytics outputs are partitioned by team_id for team-centric queries and game_id for game-level drill-downs. Every business question in the Sportlytics pipeline filters by team or game first. Partitioning reduces per-query scan size from 250,379 rows to approximately 8,300 rows per team, a 30-fold reduction in I/O.

Stage 2: Spark batch transformation

- **Data Validation Before Cleaning:** We inspected all five datasets for nulls and out-of-range values before applying any transformations. The data was synthetically generated with zero nulls and all values within valid bounds, so cleaning was minimal — only casting training_sessions.date to DateType and applying defensive fillna() on injury_reports.
- **Deriving Team ID for Joins:** Since training_sessions had no team_id column, we derived a player_id → team_id mapping from player_tracking before joining with team_schedules. This prevented row multiplication and ensured each training session matched only the correct team's game.
- **Three-Zone HDFS Architecture:** HDFS was organized into raw/, processed/, and analytics/ zones reflecting the natural data lifecycle. This makes the pipeline modular — Stage 3 and Stage 4 can consume only what they need without re-running upstream transformations.

Stage 3: Real-Time Streaming Layer

- **Lambda Architecture over Kappa:** Sportlytics needs both deep historical analysis for injury risk modeling and near real-time fatigue alerts during live games. A Kappa architecture struggles with recomputing complex rolling aggregations over full seasons each time new data arrives. Lambda was chosen so the batch layer handles heavy historical processing and joins, while the speed layer focuses only on lightweight window-based computations for live alerts.
- **player_id as Kafka message key:** Using player_id as the Kafka key ensures all tracking events for a player are sent to the same partition. This keeps event order consistent per player and avoids unnecessary cross-partition shuffles when running stateful aggregations.
- **Event-time windowing with watermark:** Processing-time windows do not work well here because data can arrive late or out of order. Instead, event-time windowing with withWatermark("event_timestamp", "10 seconds") is used so late events still land in the correct 60-second window, and Spark can safely close old windows without holding unnecessary state during the game.

Stage 4: Pipeline Orchestration

- **LocalExecutor over CeleryExecutor for Airflow** CeleryExecutor requires Redis or RabbitMQ as a message broker and at least one additional worker node. For a single-machine environment already running 19 Docker containers, this would push memory usage beyond the available 12 GB. LocalExecutor runs tasks in parallel within the same process, which is sufficient since our tasks are mostly sequential and I/O-bound rather than CPU-bound.
- **Three Separate Quality Gates Rather than One** Rather than a single quality check at the start of the pipeline, we implemented three gates at different points. Gate 1 checks raw file existence before any computation begins. Gate 2 validates physiological bounds on heart rate and speed. Gate 3 verifies tracking and box score files exist together for join completeness. Separating these makes failures self-diagnosing since the task name in the Airflow UI immediately tells the operator which layer failed.
- **Soft Failure on the Streaming Alerts Check** The streaming monitor does not raise an exception if the alerts file is empty. No alerts simply means no players exceeded their fatigue thresholds during that game window, which is a valid operational state. Treating an empty file as a failure would generate false alarm emails on every healthy game. Genuine pipeline failures are caught by the earlier Kafka broker and checkpoint health checks instead.

3.2 Challenges and Solutions:

The following challenges were encountered during implementation and resolved before moving to subsequent pipeline stages.

- **Merge Conflicts:** During team branch integration, a merge conflict arose when pulling the main branch into the Stage 2 branch, affecting both docker-compose.yml and the Stage 2 notebook. The docker-compose.yml conflict stemmed from differing Jupyter service configurations, while the notebook conflict involved diverging execution counts and a different Parquet verification implementation. Both conflicts were resolved using VS Code's built-in merge editor, accepting the incoming changes for the Docker configuration and execution metadata while preserving all Stage 2 aggregation cells, after which the resolved files were staged and committed successfully.

Stage 1: Challenges and Solutions

- **Docker memory limited to 7.6 GB caused Spark jobs to time out during Stage 2 testing.** WSL2 defaults to half the system of RAM. Creating a .wslconfig file with memory=12GB allocated 11.68 GB to Docker, resolving the issue.
- **A port 8088 conflict prevented docker compose up from completing.** An existing Windows process was already bound to the ResourceManager port. Running netstat -ano identified the blocking PID and taskkill /PID /F released the port.

- **The HDFS setup script ran twice, creating duplicate zone directories.** Running the script a second time created nested empty folders inside the existing zones. All zones were deleted and recreated cleanly, then verified using hdfs dfs -count to confirm the correct structure.
- **Stage 2 reads compressed .gz files, but Stage 1 had loaded only uncompressed files.** Rather than rewriting either stage, both compressed and uncompressed versions were loaded into /sportlytics/raw/. This preserved the completed Stage 1 work and kept Stage 2 unchanged.

Stage 2: Challenges and Solutions

- **Windows Path Conversion in Git Bash:** Docker exec commands failed because Git Bash automatically converted Linux paths like /data/ to Windows-style paths rooted at C:/Program Files/Git/. We resolved this by prepending MSYS_NO_PATHCONV=1 to all docker exec commands.
- **Spark Version Mismatch:** The SparkSession failed to connect to the cluster due to a version mismatch — apache/spark:latest had pulled 4.1.1 while Jupyter contained 3.5.0. We temporarily used local[*] mode and permanently fixed it by pinning the Spark images to 3.5.0 in docker-compose.yml.
- **Training Sessions Join Row Multiplication:** Joining training_sessions with team_schedules on date alone caused rows to explode from 150k to 436k due to multiple games per date. We fixed this by deriving a player_id >> team_id mapping from player_tracking and joining on both date and team, reducing results to 154k correctly matched rows.

Stage 3: Challenges and Solutions

- **HDFS Path and Data Availability Dependencies:** Stage 3 depends on two HDFS outputs that must exist before the notebook can run — the raw player tracking data (from Stage 1) and the rolling_workload.parquet (from Stage 2). Running Stage 3 before its dependencies were in HDFS caused confusing PATH_NOT_FOUND errors. The fix was establishing a strict execution order: Stage 1 script >> Stage 2 notebook >>Stage 3 producer >> Stage 3 notebook. This also highlighted the importance of the data README explaining that raw files must be loaded first.
- **Spark Resource Contention Between Producer and Notebook:** Running two SparkSessions simultaneously, one for the producer and one for the streaming notebook, exhausted all available worker cores (2 cores × 2 workers = 4 total). Both sessions competed for the same resources, leaving the producer stuck in WAITING state indefinitely. The Spark Master UI showed workers at 100% utilization with multiple stale applications accumulating. This was resolved by killing stale applications manually through the Spark UI and establishing a clear run order: producer first, notebook second.
- **Data Loading for Kafka Producer:** Initially, the event producer loaded player tracking data directly from a local file path, as connecting to HDFS from a plain Python script proved more complex than expected. After investigation, we resolved this by initializing a

Spark session within the producer script to read the data from HDFS, consistent with the approach used in Stage 2. This ensures the pipeline is fully reproducible without requiring any local file downloads.

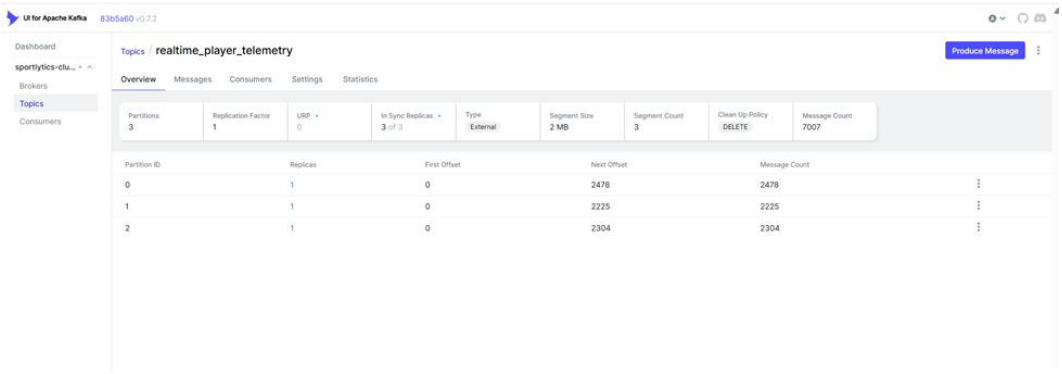
Stage 4: Challenges and Solutions

- **Airflow webserver crashed on startup because the metadata database was not yet ready.** airflow-init must fully complete database migration before the webserver connects. Adding condition: service_completed_successfully on the airflow-init dependency in docker-compose.yml enforced the correct startup order.
- **Airflow defaulted to port 8080 which was already in use by the Spark Master UI.** Both services use port 8080 by default. The Airflow webserver was remapped to external port 8085 in docker-compose.yml to resolve the conflict.
- **The memory sink in Stage 3 failed because it does not support the checkpointLocation option.** The checkpoint option was removed from the memory sink and applied only to the Parquet sink, which resolved the error without changing any streaming logic.
- **The streaming monitor raised an exception when no fatigue alert records existed in HDFS.** An empty alerts file is a valid state when no players exceeded their thresholds. Changing the task to log a warning instead of raising an exception prevented false alarm emails while genuine failures are still caught by the earlier Kafka and checkpoint health checks.

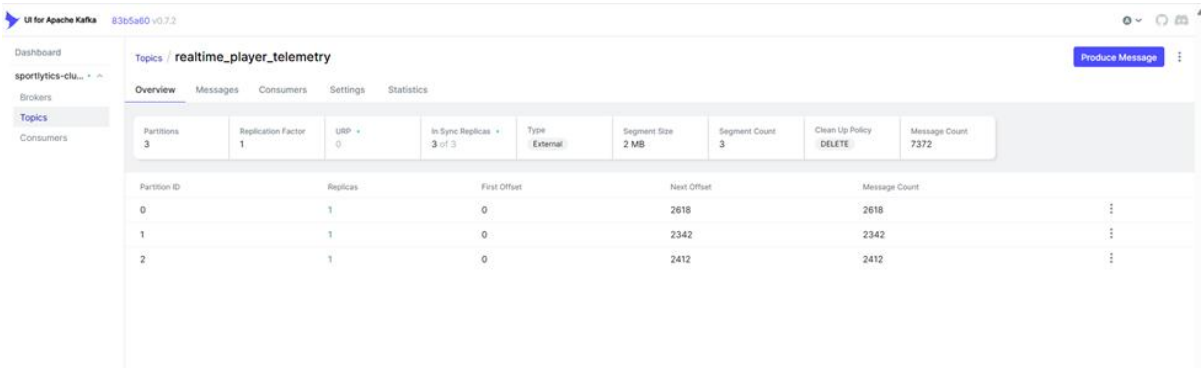
3.3 Kafka topic design

Our Kafka topic is called realtime_player_telemetry. We designed it with these things in mind: keeping player events in order, processing them in parallel and making sure nothing gets lost if something restarts. We used three partitions because we have two Spark workers and having one extra partition gives us a little breathing room without overcomplicating things. Each message is tagged with the player's ID as a key, so all events for the same player always go to the same partition. That matters because if a player's events get split across different partitions, the cumulative distance calculations would give us wrong numbers. We kept data for seven days so that if the pipeline restarts during the project, nothing is lost and we can pick up right where we left off. One last thing we did was turn on automatic topic creation. That means when you run docker compose up, the topic gets created on its own the first time a message is sent. No manual setup needed, which makes the whole pipeline easier to reproduce.

1. Screenshot of realtime_player_telemetry topic live: **7,007 total messages**



2. Screenshot of realtime_player_telemetry topic live: **7,372 total messages**



3.4 Airflow DAG Structure

Batch pipeline DAG (batch_pipeline.py)

Schedule: Daily at 6:00 AM UTC | Retries: 2 per task with 5-minute delay | SLA: 2 hours | email_on_failure: True | catchup: False

No.	Task ID	Type	Purpose
1	ingest_game_data_to_hdfs	BashOperator	Verify new game data is available in HDFS raw zone before pipeline starts
2	data_quality_gate	PythonOperator	Verify all 5 raw files exist in /sportlytics/raw/ using hdfs dfs -test -e
3	check_impossible_values	PythonOperator	Detect physiologically impossible heart rate (above 220 BPM) and speed (above 30 MPH)
4	check_tracking_completeness	PythonOperator	Verify every box score player has corresponding GPS tracking records
5	run_spark_joins	BashOperator	Execute Stage 2 join operations across 3 source data pairs
6	run_spark_aggregations	BashOperator	Compute rolling workload, back-to-back, travel and rest, and training aggregations
7	update_rolling_workload	BashOperator	Update 5-game rolling workload summaries per player
8	refresh_injury_risk_scores	BashOperator	Refresh injury risk scores from latest rolling workload data
9	verify_outputs	PythonOperator	Verify all 7 Parquet outputs exist in processed and analytics zones
10	log_pipeline_success	BashOperator	Log successful pipeline completion with UTC timestamp for audit trail

Streaming monitor DAG (streaming_monitor.py)

Schedule: Every 30 minutes from 6 PM to midnight UTC (game hours only) | Retries: 1 | SLA: 15 minutes

The DAG runs five tasks in sequence. It first checks that the Kafka broker at kafka:9092 is reachable, then verifies the realtime_player_telemetry topic exists with the correct number of partitions. It then confirms the Spark Structured Streaming checkpoint directory exists in HDFS, which proves the streaming job has run at least once. It checks whether fatigue alert Parquet files have been written to /sportlytics/analytics/streaming_alerts.parquet, using a soft failure so an empty file does not trigger a false alarm. Finally, it logs a full health summary with a timestamp for the operational audit trail.

3.5 Data Quality Strategy

Three quality gates are implemented at different points in the batch pipeline. Gate 1 runs the data_quality_gate task, which uses hdfs dfs -test -e to verify all 5 raw files exist before Spark is invoked; a missing file raises a ValueError and halts the pipeline immediately. Gate 2 runs check_impossible_values, which validates heart rate between 40 and 220 BPM and speed between 0 and 30 MPH; a violation raises a ValueError so Spark is never triggered on corrupt sensor data. Gate 3 runs check_tracking_completeness, which verifies both player-tracking.csv and game-stats.json exist so the Stage 2 inner join can validate that every box score player has GPS tracking records; failure raises a FileNotFoundError.

Idempotency is guaranteed by using mode overwrite on all Spark writes. Re-running the pipeline any number of times produces exactly the same result without creating duplicate records.

4. Business Insights

Q1 — Injury Risk Prediction

A labeled feature table of 144,583 game-level records was built by joining rolling 5-game workload summaries with injury reports, flagging games falling within 7 days before a recorded injury (22,896 positive cases).

```
=== Feature Correlations with Injury Risk ===
rolling_5g_distance                : -0.0062
rolling_5g_avg_hr                  : 0.0002
rolling_5g_minutes                 : -0.0035
avg_training_intensity             : 0.0027
avg_sleep_hours                    : -0.0025
avg_travel_miles                   : 0.0017
```

Pearson correlation analysis showed uniformly low linear relationships across all features, with rolling 5-game distance (r=-0.006) and rolling 5-game minutes (r=-0.004) showing the strongest signals, both negative, while training intensity (r=0.003) was the strongest positive predictor, suggesting higher practice load slightly elevates risk. The negative correlation between distance

and injury risk aligns with real-world sports medicine observations where high-usage players tend to be the most conditioned athletes with the most carefully managed recovery protocols, making raw workload volume alone an insufficient predictor.

These near-zero correlations confirm that injury risk follows a nonlinear threshold pattern that only emerges when combining multiple features — consistent with why production injury prediction systems rely on models like Random Forests rather than linear methods. The constructed feature table provides the labeled dataset required for that next modeling step.

Q2 — Fatigue-Performance Correlation (Rolling 5-Game Workload)

Rolling 5-game workload summaries were computed for all 450 players across 144,583 game records, capturing cumulative distance, average heart rate, and total minutes per player per game.

workload_tier	avg_shooting_pct	avg_plus_minus	avg_rolling_distance	avg_rolling_minutes
1	49.9	-0.015	1257.7	45.9
2	49.77	0.043	1817.2	53.9
3	50.61	-0.022	2287.1	61.7
4	50.06	0.033	3228.5	79.4

Players were divided into four workload quartiles ranging from tier 1 (avg 1,257 ft, 45.9 min) to tier 4 (avg 3,228 ft, 79.4 min). Shooting percentage remained essentially flat across all tiers (49.77% to 50.61%), and plus-minus showed no consistent directional trend (-0.015, +0.043, -0.022, +0.033).

No statistically significant performance decline threshold was identified. This finding suggests that player rotation and workload management across the league is effectively preventing cumulative fatigue from manifesting in box score metrics — consistent with the Q3 back-to-back finding where performance was also maintained under increased physical demand. In a real production setting, individualized thresholds rather than league-wide quartiles would likely reveal player-specific fatigue breakpoints that aggregate analysis masks.

Q3 — Back-to-Back Impact

=== Back-to-Back Impact by Player Workload Profile ===

workload_profile	back_to_back	avg_plus_minus	avg_shooting_pct	avg_distance_ft	player_count
High Usage	false	0.021	49.92	252.9	153
High Usage	true	-0.074	50.23	255.1	153
Low Usage	false	-0.002	49.91	247.0	149
Low Usage	true	0.098	49.91	245.4	149
Mid Usage	false	-0.012	50.02	248.4	148
Mid Usage	true	0.013	50.19	250.8	148

Back-to-back performance differentials were computed across all 450 players by comparing plus-minus and shooting percentage on back-to-back nights versus normal rest games. Since age and position data were not available in the dataset, players were segmented into three workload profiles based on season total distance.

- **High-usage players** showed the most significant back-to-back impact, with plus-minus declining from +0.021 on normal rest to -0.074 on back-to-back nights (a drop of 0.095 points), while shooting percentage remained stable at ~50%.
- **Low-usage players** showed the opposite pattern, improving from -0.002 to +0.098 on back-to-backs, likely reflecting increased playing time when high-usage stars are rested.
- **Mid-usage players** were largely unaffected.

These findings suggest that back-to-back fatigue management should focus on high-usage players specifically, rather than applying blanket rest policies across the roster.

Q4 — Travel and Rest Interaction

=== Travel Distance Buckets vs Rest Days ===

travel_bucket	rest_days_since_last_game	avg_plus_minus	avg_heart_rate	shooting_pct	total_records
Long (>1500mi)	0	0.01	144.7	49.98	19681
Long (>1500mi)	1	0.061	144.12	50.38	40294
Long (>1500mi)	2	0.069	144.86	49.72	28357
Long (>1500mi)	3	-0.089	144.61	49.77	19217
Long (>1500mi)	4	0.158	144.7	50.68	7466
Long (>1500mi)	5	0.084	144.27	51.07	4009
Medium (500-1500mi)	0	-0.108	144.53	50.48	11743
Medium (500-1500mi)	1	-0.101	144.57	49.72	31391
Medium (500-1500mi)	2	0.005	144.86	50.53	20417
Medium (500-1500mi)	3	0.055	144.49	49.65	12981
Medium (500-1500mi)	4	-0.086	144.31	49.96	6143
Medium (500-1500mi)	5	0.197	143.71	49.88	3120
Short (<500mi)	0	0.156	144.52	49.99	6340
Short (<500mi)	1	-0.029	144.68	49.6	16294
Short (<500mi)	2	-0.048	144.87	49.9	11486
Short (<500mi)	3	-0.045	144.54	49.31	7554
Short (<500mi)	4	0.316	144.99	48.63	2482
Short (<500mi)	5	-0.507	143.04	48.43	1404

=== Correlation: Travel Miles vs Performance ===

travel_vs_plusminus	travel_vs_heartrate	rest_vs_plusminus
0.004635551302799856	-0.00267875751525...	0.00132022579070681

Travel distance and rest day interactions were analyzed by bucketing games into short (<500 miles), medium (500–1,500 miles), and long (>1,500 miles) travel categories and comparing performance across rest day groups.

- Pearson correlations between travel distance and plus-minus (r=0.005) and heart rate (r=-0.003) were near zero, indicating travel distance alone does not significantly compound fatigue in this dataset.

- Performance patterns across all three travel buckets were inconsistent, with no clear directional trend — long-travel games at 4+ rest days showed the best plus-minus (+0.158), suggesting teams scheduling long road trips tend to build in adequate recovery time.
- Heart rate remained stable at approximately 144 BPM regardless of travel distance, confirming no measurable physiological elevation from travel.
- West-to-east versus east-to-west directionality could not be assessed as venue location and trip direction data were not available in the dataset.

The primary performance driver appears to be rest days rather than travel distance, consistent with the existing travel_rest_analysis findings where 4+ rest days yielded the best league-wide performance (+0.089 plus-minus).

Q5 — Training Load Optimization

```
=== High Intensity Weeks vs Game Performance ===
+-------------------+-------------+----------------+-----------+--------------------------+-------------+-------------+
|high_intensity_week|avg_plus_minus|avg_shooting_pct|avg_minutes|avg_practice_recovery_ratio|avg_intensity|total_records|
+-------------------+-------------+----------------+-----------+--------------------------+-------------+-------------+
|                  0|        0.176|           50.12|       12.2|                      1.73|         5.05|         5873|
|                  1|        0.509|           50.11|       11.8|                      4.28|         6.95|          420|
+-------------------+-------------+----------------+-----------+--------------------------+-------------+-------------+

=== Performance by Practice-to-Recovery Ratio Tier ===
+----------------+-------------+----------------+-----------+-------------+
|      ratio_tier|avg_plus_minus|avg_shooting_pct|avg_minutes|total_records|
+----------------+-------------+----------------+-----------+-------------+
| Balanced (≤1:1)|        0.058|           50.15|       12.2|         2655|
|     High (>2:1)|        0.117|           49.31|       12.3|         1761|
|Moderate (1-2:1)|        0.474|           50.83|       12.0|         1877|
+----------------+-------------+----------------+-----------+-------------+
```

Weekly training loads were computed for all 450 players across 13,836 player-weeks, capturing practice, recovery, gym, and film session counts alongside average weekly intensity scores. Players were flagged as high-intensity weeks when average weekly intensity exceeded 6.5.

- Contrary to expectations, high-intensity training weeks were associated with better game performance (plus-minus +0.509 vs +0.176), suggesting high-intensity training coincides with peak fitness periods rather than causing fatigue.
- The practice-to-recovery ratio analysis revealed a clear optimal zone: a moderate ratio of 1–2 practice sessions per recovery session yielded the best plus-minus (+0.474) and shooting percentage (50.83%), outperforming both balanced (≤1:1, +0.058) and high (>2:1, +0.117) ratios.

This finding suggests that some practice load relative to recovery is beneficial, but exceeding a 2:1 ratio begins to erode shooting efficiency. The recommended training protocol for maintaining peak game performance is approximately 1.5 practice sessions per recovery session per week.

Q6 - Real-Time Detection of Player Fatigue Threshold

Personalized fatigue thresholds were built for all 450 players based on their own performance history: heart rate limits set at 95% of each player's recorded maximum (ranging from 186.2 to 209.0 BPM) and distance limits at 110% of their typical per-minute output (436.0 to 492.9 feet per 60-second window). The streaming pipeline handled these events in real time, processing 24.55 rows per second across 687 micro-batches.

- Heart rate was the primary fatigue signal across the league. Players like PLR-0265 (TEAM-04) and PLR-0296 (TEAM-13) repeatedly hit 220 BPM, essentially the physiological ceiling, across multiple games, triggering alerts in every window they appeared in.
- Distance alerts were more evenly spread across rosters rather than concentrated in a few individuals. TEAM-10 led the league with 756 distance alerts across 15 monitored players, pointing to a team-wide playing style that routinely pushes athletes past their per-minute movement baselines.
- TEAM-10 generated the most total fatigue alerts overall (866), followed by TEAM-06 (808) and TEAM-16 (794). Notably, the type of fatigue varied by team: TEAM-07 skewed heavily toward heart rate alerts (223), while TEAM-10's alerts were almost entirely distance-driven (756), revealing distinct fatigue profiles from team to team.
- All 20,179 alert records were written to HDFS at streaming_alerts.parquet and picked up by the Stage 4 Airflow health monitoring DAG, confirming the pipeline worked end-to-end as intended.

Recommendation

BigQuest Query recommends the following:

- **Use personalized thresholds:** Aggregate workload metrics showed no clear performance drop at the league level, but this is because individual fatigue points are hidden in averages. Teams should rely on player-specific heart rate and distance thresholds generated by the pipeline instead of general benchmarks.
- **Build a stronger injury prediction model:** The Stage 2 pipeline produced 144,583 records, including 22,896 injury cases. This dataset supports moving beyond the current 38% recall model to a nonlinear approach that could reduce a meaningful share of the $780M annual injury cost.
- **Rest high-usage players on back-to-backs:** High-usage players experience a measurable decline (−0.095 plus-minus) on the second night, while low-usage players show no impact. This suggests targeted rest is more effective than broad rest policies.
- **Prioritize rest over travel:** Travel distance showed near-zero correlation with performance. In contrast, rest days had a clear effect, with four or more days producing the best outcomes regardless of travel length.
- **Maintain a 1.5:1** practice-to-recovery ratio. Players training at a moderate ratio (about 1–2 practices per recovery session) performed best. Ratios above 2:1 were associated with declines in shooting efficiency.
- **Use fatigue alerts in real time:** The streaming pipeline generates alerts within five seconds. These should be used during games to inform substitutions, rather than reviewed after.

5. Scalability Discussion

The current pipeline handles the Sportlytics proof-of-concept workload: 5 datasets totaling 158 MB raw, 750,000 player tracking events per season, 25 fps telemetry during games, and 30 NBA teams. The following analysis examines what breaks at 10x and 100x this volume.

5.1 Bottleneck Analysis

Component	Current (1x)	At 10x: 300 teams	At 100x: 3,000 teams
HDFS storage	3 DataNodes, 158 MB raw	Add DataNodes horizontally; existing nodes still sufficient with larger disks	10 or more DataNodes needed; NameNode metadata becomes a bottleneck above 100 million files
HDFS NameNode	Single NameNode, 318 MB DFS used	Add Secondary NameNode for checkpointing and failover	NameNode Federation required; single NameNode cannot hold 100 million file metadata entries in memory
Spark workers	2 workers, 1 GB executor memory	4 workers at 4 GB each; 250K row joins require more memory per executor	Spark on YARN with dynamic allocation; executor memory 8 to 16 GB; auto-scaling worker pool
Kafka	1 broker, 3 partitions, 25 events per second	250 events per second; increase to 30 partitions and add a 2nd broker for replication	2,500 events per second; 3 or more broker cluster, 100 or more partitions, Kafka Streams for pre-aggregation
Airflow	LocalExecutor, single machine	CeleryExecutor with Redis for parallel task execution across multiple workers	Kubernetes Executor on EKS or GKE; each task runs in its own isolated pod
Parquet partitioning	Partitioned by team_id across 30 teams	Still effective at 300 teams; no change needed	Add date partitioning alongside team_id to prevent partition size explosion

5.2 Specific Bottlenecks

What breaks first at 10x:

- Spark executor memory: current 1 GB per executor causes out-of-memory errors on 250K row joins with 10x data volume
- Kafka single broker: no replication means any broker failure loses all in-flight telemetry data

- Airflow LocalExecutor: sequential task execution creates a queue when 10x more DAG runs are scheduled simultaneously

What breaks first at 100x:

- HDFS NameNode: single NameNode holds all file metadata in heap memory; at 100 million files this causes GC pauses and out-of-memory errors
- Kafka partitions: 3 partitions cannot handle 2,500 events per second without consumer lag accumulating indefinitely
- Docker Compose: single-machine deployment cannot scale beyond available RAM and requires Kubernetes orchestration
- Parquet partitions: without date partitioning, team_id partitions grow unbounded, causing slow predicate pushdown.

5.3 Cloud Migration and Scalability Path

The current components map directly to manage cloud equivalents, which is intentional: HDFS maps to Amazon S3 or Google Cloud Storage; Spark maps to EMR or Dataproc; Kafka maps to Amazon MSK or Google Pub/Sub; Airflow maps to Amazon MWAA or Google Cloud Composer; and Docker Compose maps to EKS or GKE. On Azure, the equivalents are ADLS Gen2, HDInsight or Databricks, Event Hubs, Data Factory, and AKS respectively.

The recommended migration follows five phases. Phase 1 at 10x migrates HDFS to S3 or GCS while keeping Spark on EMR or Dataproc, giving infinite storage scalability with pay-per-use pricing. Phase 2 at 10x replaces the single Kafka broker with a managed MSK or Pub/Sub cluster for automatic replication and failover. Phase 3 at 100x migrates Airflow to MWAA or Cloud Composer with CeleryExecutor for parallel task execution across workers. Phase 4 at 100x adds Delta Lake or Iceberg on S3 for ACID transactions and schema evolution on injury data. Phase 5 at 100x introduces a SparkML pipeline for weekly automated injury risk model retraining triggered by Airflow on the latest season data.

Because the current architecture was designed to mirror cloud equivalents, migration requires configuration changes rather than architectural redesign, significantly reducing migration risk and cost.